
Correcting Within-Group Self-Selection Bias in Prioritized Replay

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Prioritized experience replay (PER) improves sample efficiency by replaying high-
2 priority transitions, usually according to absolute temporal-difference error. In
3 stochastic environments, this transition-level rule has a conditional side effect:
4 among replay-buffer transitions that share the same state-action pair, PER changes
5 the empirical distribution of realized outcomes seen by the learner. We call this
6 within-group self-selection. We formalize PER as a product of a between-group
7 allocation over state-action groups and a conditional distribution over sibling out-
8 comes, and derive corrections that preserve group-level prioritization while replac-
9 ing priority-weighted sibling selection with empirical sibling sampling: SAMPLE
10 selects a group through PER and then trains on a uniformly sampled sibling;
11 AVG averages sibling Bellman targets; and MODEL samples from an empirical
12 full-outcome model. Our empirical evaluation shows that, in exact state-action
13 environments with rare high-magnitude outcomes, PER develops low effective
14 sample size, large TD-error tails, and poor learning efficiency, while sibling-aware
15 replay improves these diagnostics and downstream return or success. In the Mi-
16 nAtar function-approximation benchmark, approximate VQ-VAE sibling groups
17 combined with SAMPLE mitigate PER degradation under mean-preserving reward
18 tails in four of five games. The results support our claim that some PER failures
19 arise not from revisiting the wrong state-action regions, but from prioritizing the
20 wrong outcome realizations inside the right regions.

21 1 Introduction

22 Experience replay stores past transitions and reuses them for updates, reducing temporal correlation
23 and improving data reuse in off-policy reinforcement learning [11]. In deep value learning, DQN
24 combined replay with target networks to stabilize training from high-dimensional observations [12],
25 while prioritized experience replay (PER) changed the replay distribution by sampling transitions
26 according to priority, typically a power of absolute temporal-difference (TD) error [15]. Priority-
27 driven replay has since become part of high-performing value-based agents such as Rainbow, Ape-X,
28 and R2D2 [4, 5, 7]. More recent analyses show that replay capacity, replay ratio, and buffer
29 composition can substantially change learning outcomes [21, 3]. For PER, this makes the unit of
30 prioritization important: transition-level priorities can change both which state-action regions are
31 revisited and which stochastic outcomes dominate within those regions.

32 Non-uniform replay also changes the distribution of updates. PER samples individual transitions
33 according to priority and applies importance-sampling (IS) weights to correct the resulting sampling
34 bias [15]. In practice this correction is partial: the IS exponent is often annealed rather than fixed
35 at one, weights are normalized inside minibatches, priorities become stale, and targets move with
36 the online and target networks [15, 14, 9]. Prior work has therefore identified several limitations

of transition-level prioritization. Some analyses study the effective loss, coverage, and gradient-estimation consequences of non-uniform replay [14, 9, 10]; others change the priority score to reduce replay of samples whose high errors are driven by irreducible or aleatoric noise [16, 2]. Together, these works show that transition-level priority is useful but imperfect as a proxy for learning value.

This paper isolates a conditional form of replay distortion. Consider a replay buffer containing several transitions with the same state-action pair (s, a) but different realized stochastic outcomes. We call these transitions *siblings*. PER performs two operations at once. First, it allocates more replay mass to some state-action groups than others; we call this *between-group allocation*. Second, within a selected group, it changes which realized outcomes are replayed more often; we call this *within-group self-selection*. Prior work mainly changes the transition priority score, and therefore changes the overall transition distribution or the amount of replay mass assigned to different regions. We instead keep the group-level priority mass and target the second effect: the conditional outcome frequencies used once a group has already been selected.

The first effect is often the desired behavior of PER, since some state-action regions may deserve more updates. The second effect can be harmful. If rare high-magnitude outcomes have persistently large TD errors for aleatoric reasons, then PER can replay the right region at the wrong conditional outcome frequencies. In that case, the problem is not only that some transitions receive too much replay probability globally, but that the empirical outcome distribution inside a fixed state-action group is tilted toward high-priority realizations.

We propose sibling-aware replay methods that separate PER’s group-level and within-group effects. The group-level priority mass is preserved, so high-priority state-action regions can still receive more replay effort. Within a selected group, the update is instead constructed from the empirical sibling distribution. SAMPLE trains on a uniformly sampled sibling transition; AVG averages sibling Bellman targets; and MODEL samples from the empirical full-outcome distribution for the exact state-action pair. Our contributions are: 1) We formalize a finite-buffer decomposition of PER into priority mass over state-action groups and priority-weighted sibling selection within each group. 2) We derive the induced sampling distribution and group-aware importance weights for SAMPLE, and we define AVG and MODEL as empirical estimators of the within-group Bellman target. 3) We evaluate the resulting methods in exact stochastic environments, where sibling groups are exact state-action groups, and in a function-approximation setting where frozen VQ-VAE codes provide approximate latent groups.

2 Related work

Experience replay and prioritized value learning. Replay was introduced as a mechanism for reusing past experience in reinforcement learning [11], and later became a core stabilizer of deep value learning through DQN [12]. Prioritized experience replay samples transitions according to priority, typically based on absolute TD error, and uses importance weights to correct the resulting non-uniform sampling distribution [15]. Priority-driven replay has since been incorporated into strong value-based agents such as Rainbow, Ape-X, and R2D2 [4, 5, 7]. Complementary analyses show that replay capacity, replay ratio, and buffer composition can substantially change learning outcomes [21, 3]. These works motivate studying replay as an algorithmic sampling distribution. We analyze a conditional distortion that arises inside priority-weighted transition replay.

Analyses of prioritized replay. PER is usually formalized as a distribution over individual replayed transitions, with importance sampling correcting toward uniform replay [15]. Pan et al. [14] show that error-based prioritized sampling can be interpreted as optimizing a different loss and highlight stale priorities and insufficient coverage as limitations. LaBER formulates replay sampling as a gradient-estimation problem [9]. A value-function perspective analyzes when TD error can proxy the utility of replaying a transition [10]. These approaches ask which individual transitions should receive more replay probability, or how transition priorities should be corrected. We ask a different question: after priority mass has selected a state-action group, which realized outcome from that group should define the update?

Noisy, irreducible, and aleatoric transitions. Several recent methods address the fact that large TD error need not correspond to useful learning signal. ReLo distinguishes reducible from irreducible loss and prioritizes samples whose losses are expected to decrease [16]. UPER uses epistemic uncertainty to avoid transitions whose high errors are dominated by aleatoric randomness [2]. These are the

closest conceptual neighbors to our work because they also study when high-error transitions are not useful replay targets. However, they address this issue by changing the priority score assigned to transitions, thereby changing the overall priority distribution across transitions or groups. Our method does not classify errors as reducible or irreducible. It keeps the existing priority mass over groups and corrects only the conditional sibling distribution inside each selected group. The approaches are complementary: uncertainty-aware priorities could improve which groups receive replay effort, while sibling-aware sampling repairs the outcome frequencies used within those groups.

3 Finite-buffer decomposition of PER

Let a replay buffer contain N transitions

$$\mathcal{D} = \{z_i\}_{i=1}^N, \quad z_i = (s_i, a_i, r_i, s'_i, d_i), \quad (1)$$

where d_i denotes a terminal indicator. Let $p_i > 0$ be the raw priority, let $\alpha \geq 0$ be the priority exponent, and define the sampling mass $u_i = p_i^\alpha$. PER samples transition i with

$$P_{\text{PER}}(i) = \frac{u_i}{S}, \quad S = \sum_{k=1}^N u_k, \quad (2)$$

where S is the total priority mass in the buffer. For fixed priorities and a fixed per-transition loss $\ell_i(\theta)$, the idealized expected objective under unnormalized IS weights $w_i = (NP_{\text{PER}}(i))^{-\beta}$ is

$$\mathcal{J}_{\text{PER},\beta}(\theta) = \sum_{i=1}^N P_{\text{PER}}(i) w_i \ell_i(\theta) \propto \sum_{i=1}^N u_i^{1-\beta} \ell_i(\theta), \quad (3)$$

where $\beta \in [0, 1]$ controls the strength of the correction and the omitted factor is independent of θ . Thus $\beta = 1$ recovers the uniform-buffer objective in this fixed setting, while $\beta < 1$ leaves residual priority tilt. In deep RL this idealized statement is further complicated by moving targets, stale priorities, and minibatch weight normalization [15, 14, 9].

Now define a fixed group key $\kappa(i)$. In the exact tabular setting, $\kappa(i) = (s_i, a_i)$. A group g contains the sibling indices

$$I(g) = \{i : \kappa(i) = g\}, \quad n_g = |I(g)|, \quad s_g = \sum_{j \in I(g)} u_j. \quad (4)$$

Here n_g is the number of observed siblings in group g , and s_g is their total priority mass. The PER sampling distribution decomposes as

$$P_{\text{PER}}(\kappa = g) = \frac{s_g}{S}, \quad P_{\text{PER}}(i \mid \kappa = g) = \frac{u_i}{s_g}, \quad i \in I(g). \quad (5)$$

Equation (5) separates two phenomena. The first term is PER's between-group allocation: some state-action groups receive more updates. The second is within-group sibling selection: once a group is revisited, high-priority realized outcomes are over-sampled relative to their empirical frequency. The empirical conditional sibling distribution is instead $P_{\mathcal{D}}(i \mid \kappa = g) = 1/n_g$.

The same distinction appears in the update target. For any fixed scalar target T_i associated with transition i , the empirical sibling target for group g is the uniform average

$$\bar{T}_g = \frac{1}{n_g} \sum_{i \in I(g)} T_i, \quad (6)$$

whereas PER emphasizes the priority-weighted conditional target

$$\tilde{T}_g^{\text{PER}} = \sum_{i \in I(g)} \frac{u_i}{s_g} T_i. \quad (7)$$

We call the gap between u_i/s_g and $1/n_g$ *within-group self-selection*. It is a finite-buffer sampling property, not by itself a claim about the true environment distribution. Uniform sibling sampling exactly recovers the empirical conditional distribution over observed siblings; it approximates the environment conditional distribution only insofar as the buffer is representative. The pathology appears when priority is driven by rare stochastic outcomes rather than reducible learning error: PER then biases the update toward high-priority realizations inside the group, rather than the empirical outcome frequencies.

125 **Scope.** All exact statements below assume a fixed buffer, fixed priorities, and fixed target parameters.
 126 They characterize the stochastic update distribution induced by replay and do not prove convergence
 127 of nonlinear Q-learning. The decomposition applies to any fixed key map κ ; our exact claims use
 128 $\kappa(i) = (s_i, a_i)$, while the function-approximation experiments later use latent keys of the form
 129 $\kappa_\phi(i) = (\phi(s_i), a_i)$ for a frozen abstraction ϕ . Latent keys correct only within the chosen abstraction
 130 and can introduce abstraction error.

131 4 Sibling-aware replay

132 The methods below preserve PER’s group-selection role but change the outcome used inside the
 133 selected group. Algorithmically, PER samples an anchor transition A , the group $g = \kappa(A)$ is
 134 identified, and the within-group rule constructs the transition or target used in the update. Appendix B
 135 gives the corresponding finite-buffer derivations.

136 4.1 SAMPLE: uniform sibling replacement

137 SAMPLE treats the PER anchor only as a group selector. After drawing $A \sim P_{\text{PER}}$, it samples the
 138 trained index J uniformly from the anchor’s sibling group $I(\kappa(A))$ and trains on the full transition
 139 z_J . Thus, for any trained index j , writing $g = \kappa(j)$,

$$q_{\text{SAMPLE}}(j) = \Pr(J = j) = \sum_{i \in I(g)} \frac{u_i}{S} \frac{1}{n_g} = \frac{s_g}{Sn_g}. \quad (8)$$

140 The finite-buffer IS ratio from this sampler back to uniform replay is

$$\rho_j = \frac{1/N}{q_{\text{SAMPLE}}(j)} = \frac{n_g S}{N s_g}, \quad w_j^{\text{grp}} = \rho_j^\beta. \quad (9)$$

141 At $\beta = 1$, fixed priorities, and fixed per-transition losses, SAMPLE with group-aware IS matches
 142 the uniform-buffer estimator. For $\beta < 1$, residual priority tilt remains across groups through s_g ,
 143 but siblings are uniform within each selected group. This is the intended compromise: retain
 144 priority-shaped state-action revisitation while removing priority-shaped outcome selection.

145 4.2 AVG and MODEL

146 In our value-based experiments, let

$$Y_j = r_j + \gamma(1 - d_j)Q_{\bar{\theta}}(s'_j, \arg \max_a Q_{\theta}(s'_j, a)) \quad (10)$$

147 be the Double-DQN target for sibling j [19]. AVG is a target-level correction: it replaces the sampled
 148 target by an average of sibling Bellman targets,

$$\hat{Y}_{g,K} = \frac{1}{K_g} \sum_{j \in U_K(g)} Y_j, \quad K_g = \min(K, n_g), \quad (11)$$

149 where $U_K(g)$ is a uniformly sampled subset of K_g siblings from $I(g)$. With the full group, AVG
 150 gives the empirical conditional Bellman target for g ; with a uniform subset, it is an unbiased finite-
 151 population estimator of that target.

152 MODEL instead samples from an empirical full-outcome distribution for each exact state-action group.
 153 It stores counts over tuples

$$Z = (s', r, d_{\text{term}}, d_{\text{trunc}}) \quad (12)$$

154 and samples a full outcome tuple when the corresponding exact group is selected. Sampling the
 155 joint tuple preserves dependencies between reward, next state, termination, and truncation. MODEL
 156 is directly applicable only in settings where exact state-action groups and full outcome tables are
 157 meaningful.

4.3 Approximate latent siblings

The exact methods use $\kappa(i) = (s_i, a_i)$. For continuous, high-dimensional discrete, or image-based domains, exact state equality is usually too sparse or semantically brittle to define useful sibling groups. We therefore also consider a fixed abstraction ϕ and define latent keys

$$\kappa_\phi(i) = (\phi(s_i), a_i). \quad (13)$$

In the function-approximation setting considered here, ϕ is a frozen VQ-VAE encoder trained on random-policy observations [18]. It maps each observation to a packed discrete code grid. The same finite-buffer identities apply relative to these fixed abstract groups, but the correction no longer corresponds to the exact conditional outcome distribution for a true state-action pair. We select the abstraction using support diagnostics: code usage, observed $(\phi(s), a)$ buckets, bucket-size quantiles, fraction of transitions in sufficiently supported buckets, next-code entropy, and reward variability. Appendix D and Table 4 report the support diagnostics and selected abstractions.

5 Experiments

We evaluate two claims. First, in exact state-action environments, PER’s conditional sibling selection should create identifiable replay pathologies under rare stochastic outcomes, and sibling-aware replay should reduce them. Second, when exact state equality is not useful, approximate sibling groups should still mitigate the same failure mode under function approximation. We report evaluation return and success, normalized area under the evaluation curve (AUC), late-window performance, effective sample size (ESS), TD-error dispersion, and replay-policy action-frequency divergence. AUC summarizes learning efficiency over training; late-window metrics average the final evaluation window. ESS is computed per minibatch from the update weights as $(\sum_b w_b)^2 / \sum_b w_b^2$; it equals the batch size under uniform weights and decreases as replay concentrates updates. Exact-environment results use ten seeds, and MinAtar results use five seeds.

Exact environments. We use two custom diagnostic environments and one standard gridworld family. OutlierBandit is a one-state two-action bandit designed to isolate rare-reward over-sampling: a safe action gives reward 2, while a risky action gives reward 100 with probability 0.01 and 0 otherwise. The risky action has lower expected value, but its rare high reward can produce large TD errors. TwoChains uses the same idea, formulated as two branches: the safe branch gives deterministic reward 1, while the risky branch gives reward 45 with probability 0.02. FrozenLake is based on the Gym/Gymnasium gridworld interface [1, 17]: the agent attempts to move on a grid, moves in the intended direction with some probability, and otherwise slips to another direction. We use an 8×8 slippery variant with intended-action probability 0.99, step reward -0.01 , and modified terminal rewards to create controlled high-magnitude outcome tails. FrozenLake-H100/H300 uses goal reward $+100$ and hole reward -300 as the main exact gridworld experiment; FrozenLake-H50 uses $+50 / -100$ as a milder setting.

MinAtar. MinAtar provides compact Atari-like visual environments with lower computational cost than full Atari [20]. We use Asterix, Breakout, Freeway, Seaquest, and SpaceInvaders. Clean MinAtar is a control condition. The tail condition is MP-TAIL: for every positive training reward r , we replace it by $r - 30$ with probability 0.02 and by $r + 0.02 \cdot 30 / 0.98$ otherwise. This preserves the expected reward while introducing rare high-magnitude negative outcomes. Evaluation is always performed on the unperturbed environment. Latent SAMPLE uses frozen VQ-VAE state codes paired with actions as approximate sibling keys.

5.1 Exact state-action groups

Figure 1 and Table 1 show the exact-group results. In OutlierBandit, all methods eventually learn the safe action, but PER has substantially lower AUC because it spends more training time attracted to rare risky outcomes, while DDQN and SAMPLE recover the safe action more quickly. In TwoChains, the same distortion affects the learned branch choice: PER has poor safe-branch success, while SAMPLE and MODEL recover much stronger performance. These environments therefore isolate the mechanism in settings where the exact sibling groups are known.

FrozenLake-H100/H300 follows the same pattern in a larger exact gridworld. The return curves in Figure 2 show that PER learns more slowly and remains below the sibling-aware methods late in

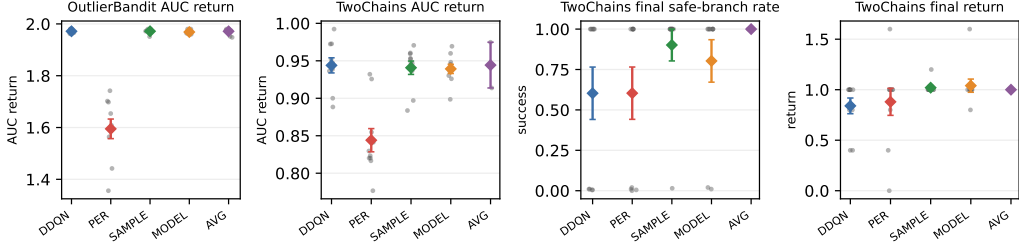


Figure 1: Exact state-action toy checks. PER lags in OutlierBandit AUC despite eventually solving the final policy, and in TwoChains its rare-tail attraction appears as poor safe-branch success. Grey points show individual seeds; colored points and error bars show mean $\pm$ standard error over ten seeds.

Table 1: Exact-environment summary, mean $\pm$ standard error over ten seeds. For OutlierBandit and TwoChains, the final columns are final evaluation metrics. For FrozenLake-H100/H300, the final columns are late-window metrics over the final 25k environment steps.

Env.	Method	AUC ret.	AUC succ.	Late/final ret.	Late/final succ.
Bandit	DDQN	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
Bandit	PER	1.59 ± 0.04	0.61 ± 0.03	2.00 ± 0.00	1.00 ± 0.00
Bandit	SAMPLE	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
Bandit	MODEL	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
TwoChains	DDQN	0.94 ± 0.01	0.66 ± 0.05	0.84 ± 0.08	0.60 ± 0.16
TwoChains	PER	0.84 ± 0.02	0.19 ± 0.06	0.88 ± 0.13	0.60 ± 0.16
TwoChains	SAMPLE	0.94 ± 0.01	0.68 ± 0.03	1.02 ± 0.02	0.90 ± 0.10
TwoChains	MODEL	0.94 ± 0.01	0.66 ± 0.03	1.04 ± 0.07	0.80 ± 0.13
FrozenLake	DDQN	8.81 ± 2.99	0.44 ± 0.02	67.71 ± 3.83	0.863 ± 0.015
FrozenLake	PER	-8.81 ± 4.69	0.15 ± 0.03	55.72 ± 7.58	0.617 ± 0.075
FrozenLake	SAMPLE	4.69 ± 3.86	0.40 ± 0.02	72.10 ± 3.71	0.871 ± 0.016
FrozenLake	MODEL	12.89 ± 3.54	0.43 ± 0.02	72.08 ± 5.17	0.883 ± 0.018
FrozenLake	AVG-g2	2.64 ± 4.13	0.36 ± 0.03	69.22 ± 4.10	0.832 ± 0.021

training. Table 1 shows that SAMPLE and MODEL recover much of the lost late-window return and success relative to PER. Compared with DDQN, SAMPLE has higher mean late return and similar late success while still using prioritized group selection; the recovery is therefore not simply uniform replay replacing PER.

Figure 4 shows the internal statistics most directly tied to the proposed mechanism. Across the exact rare-tail environments, PER concentrates updates into smaller effective batches and produces much larger sampled TD-error tails. The TD-error maxima in FrozenLake-H100/H300 track the scale of the catastrophic hole reward, while sibling-aware methods reduce these tails and restore ESS toward stable update regimes. This repeated low-ESS, high-TD-error pattern is consistent with PER over-selecting rare sibling outcomes, and the pattern is reduced when the sibling selection rule is changed.

5.2 Approximate siblings in MinAtar

Table 2 reports the MP-TAIL AUC results, while Figure 5 compares clean and MP-TAIL learning curves. Under MP-TAIL, latent SAMPLE improves over PER in Asterix, Freeway, Seaquest, and SpaceInvaders. The effect is largest in Freeway and SpaceInvaders, where PER largely fails under the tail perturbation while SAMPLE remains close to the stronger baselines. In Breakout, PER and SAMPLE are close on AUC and PER has the higher final return. This boundary case is consistent with the idea that sibling-aware replay targets tail-induced conditional skew rather than replacing PER in all settings.

Clean MinAtar acts as a control and is shown in the top row of Figure 5. Without the tail perturbation, PER often remains competitive: it is close to or above DDQN in Breakout and Seaquest, while DDQN is stronger in Freeway and SpaceInvaders. SAMPLE is similarly not uniformly dominant

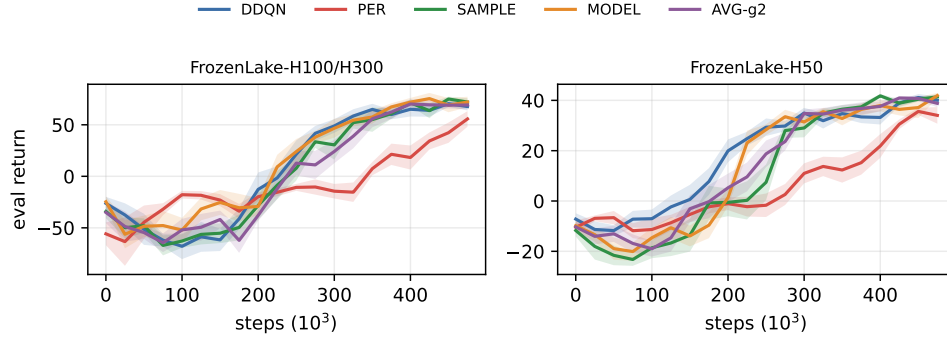


Figure 2: FrozenLake evaluation return curves. H100/H300 is the main exact gridworld test; H50 is a milder robustness setting. Shaded regions show standard error over ten seeds.

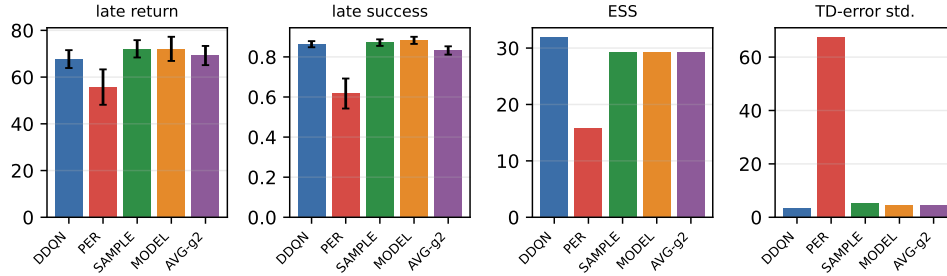


Figure 3: FrozenLake-H100/H300 late-window metrics, mean $\pm$ standard error over ten seeds. Sibling-aware replay repairs PER’s low ESS and TD-error explosion while matching or exceeding DDQN late performance.

on clean games. The degradation appears in the bottom row, where the mean-preserving tail makes high-magnitude stochastic outcomes frequent in replay while leaving expected rewards unchanged. This is the setting where we expect transition-level priorities to be most problematic.

DDQN is a strong baseline, but under MP-TAIL, the paired SAMPLE–DDQN AUC differences are positive in Asterix, Breakout, and SpaceInvaders, close in Seaquest, and negative in Freeway. In the exact environments, SAMPLE also matches or improves DDQN on several late-window and AUC metrics while retaining non-uniform group selection. Therefore, our conclusion is not that SAMPLE uniformly dominates DDQN, but that it can preserve useful non-uniform replay while avoiding PER’s pathological within-group outcome selection.

5.3 Replay diagnostics beyond return

Figure 6 compares late-window ESS, TD-error standard deviation, and replay-policy action-frequency divergence. DDQN uses uniform replay by construction, whereas PER produces concentrated replay, larger TD-error dispersion, and stronger replay-policy skew. SAMPLE reduces the harmful concentration and TD-error instability while retaining more non-uniform replay than DDQN. Thus, even when its return curves resemble DDQN, SAMPLE does not collapse to uniform replay: it changes the within-group outcome selection while preserving priority-induced group selection.

6 Discussion and conclusion

We studied a conditional failure mode of PER: transition-level priorities can over-select rare stochastic outcomes inside a state-action group, even when the group itself is worth revisiting. The finite-buffer decomposition separates these two effects into group-level allocation and within-group sibling selection. Sibling-aware replay uses this separation to preserve PER’s group-level replay effort while changing the conditional outcome distribution used for updates. Across exact rare-tail environments

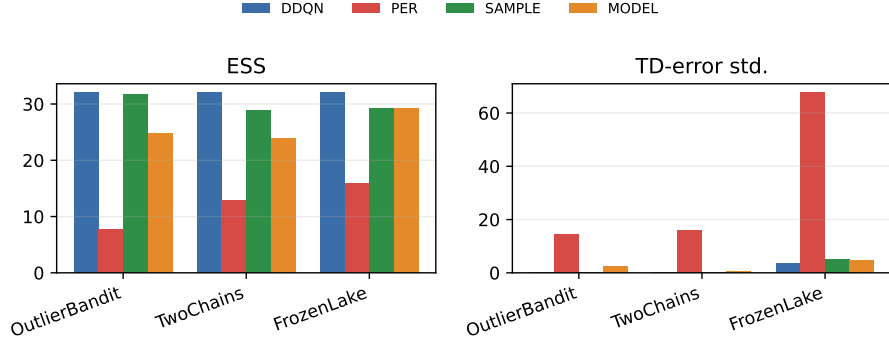


Figure 4: Exact-environment replay diagnostics, mean $\pm$ standard error over ten seeds. PER concentrates updates into smaller effective batches and produces larger sampled TD-error tails; sibling-aware replay moves those statistics back toward stable updates.

Table 2: MinAtar MP-TAIL AUC return, mean $\pm$ standard error over five seeds. Last columns report paired differences relative to SAMPLE, so positive values indicate that SAMPLE has higher AUC.

Game	DDQN	PER	SAMPLE	S-PER	S-DDQN
Asterix	22.07 $\pm$ 0.75	6.69 $\pm$ 0.39	22.80 $\pm$ 0.64	+16.11 $\pm$ 0.78	+0.73 $\pm$ 0.50
Breakout	13.72 $\pm$ 0.14	14.69 $\pm$ 0.23	14.56 $\pm$ 0.21	-0.12 $\pm$ 0.14	+0.85 $\pm$ 0.31
Freeway	49.14 $\pm$ 0.49	1.80 $\pm$ 0.10	38.57 $\pm$ 0.90	+36.77 $\pm$ 0.89	-10.57 $\pm$ 0.94
Seaquest	26.74 $\pm$ 1.55	2.12 $\pm$ 0.12	25.10 $\pm$ 2.39	+22.98 $\pm$ 2.37	-1.64 $\pm$ 2.71
SpaceInv.	73.87 $\pm$ 1.72	19.62 $\pm$ 0.63	75.64 $\pm$ 0.67	+56.02 $\pm$ 1.04	+1.77 $\pm$ 1.51

and approximate MinAtar experiments, PER develops the predicted low-ESS and high-TD-error signature, while sibling-aware replay reduces this diagnostic pattern and often recovers learning performance.

The exact contribution is a finite-buffer characterization of the replay-induced update distribution. It assumes a fixed buffer, fixed priorities, and fixed target parameters; it does not prove convergence of nonlinear Q-learning. Stale priorities, insufficient coverage, exploration failures, optimizer sensitivity, and abstraction error remain separate issues. The methods should therefore be read as targeted corrections for conditional outcome selection, not as universal replacements for all PER variants.

SAMPLE is the most practical variant. It is cheap, preserves outcome variability, and works with exact or latent groups. MODEL is strong in exact discrete environments because it samples a joint empirical outcome tuple, but it requires a faithful outcome table. AVG is theoretically natural and has lower conditional target variance for fixed targets, but the current deep-RL implementation is more sensitive to group size and priority-refresh details. In FrozenLake, AVG-g2 is useful but AVG-g4 is weaker; this is consistent with bootstrapping and priority dynamics adding effects beyond the simple fixed-target variance calculation.

For the latent MinAtar extension, VQ-VAE groups may merge observations with different conditional outcome laws or fragment observations that should share outcomes. This abstraction error is why the exact environments remain the central test of the paper. MinAtar shows that the same idea can scale empirically when exact groups are unavailable, especially under MP-TAIL, but it is not a guarantee for arbitrary latent abstractions.

Breakout under MP-TAIL does not show a large advantage of SAMPLE over PER, and clean MinAtar does not produce uniform gains. This is expected: if stochasticity is mild, if groups are mostly singleton, or if high-priority siblings correspond to reducible under-fitting rather than aleatoric outcomes, then uniformizing siblings may have little benefit. However, when transition-level priorities over-select rare outcomes inside a fixed group, sibling-aware replay directly targets that conditional distortion. Future work should learn abstractions for sibling groups rather than relying on frozen encoders, and combine sibling-aware sampling with uncertainty-aware priorities that improve the group-level allocation itself.

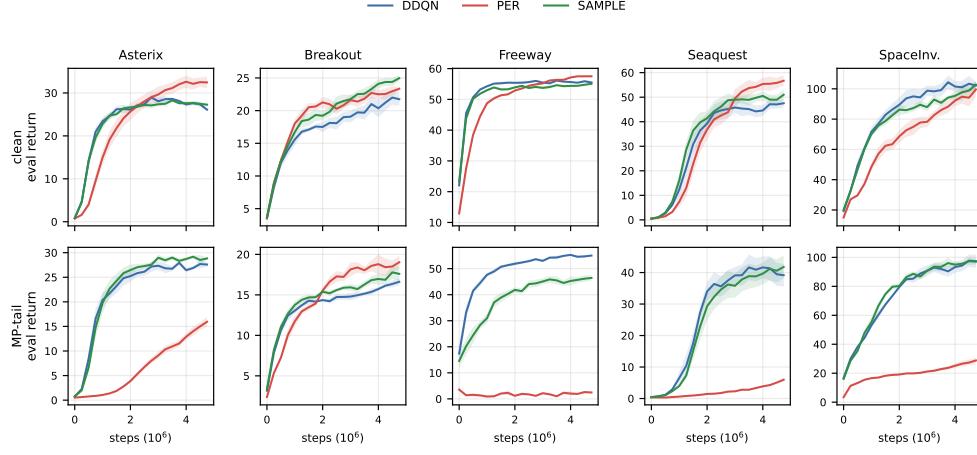


Figure 5: MinAtar evaluation return curves for clean training (top) and MP-TAIL training (bottom). Evaluation always uses the unperturbed environment; the mean-preserving tail is applied only during training. Shaded regions show standard error over five seeds.

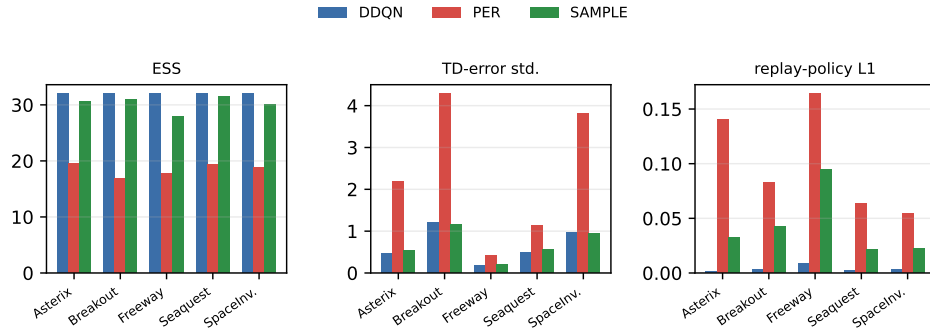


Figure 6: MinAtar MP-TAIL diagnostics, mean $\pm$ standard error over five seeds. SAMPLE restores ESS and TD-error stability toward DDQN but retains larger replay-policy skew than DDQN, showing that it has not collapsed to uniform replay.

References

- [1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016. URL <https://arxiv.org/abs/1606.01540>.
- [2] Rodrigo Antonio Carrasco-Davis, Sebastian Lee, Claudia Clopath, and Will Dabney. Uncertainty prioritized experience replay. *Reinforcement Learning Journal*, 6:585–623, 2025. URL <https://rlj.cs.umass.edu/2025/papers/Paper45.html>.
- [3] William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney. Revisiting fundamentals of experience replay. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3061–3071. PMLR, 2020. URL <https://proceedings.mlr.press/v119/fedus20a.html>.
- [4] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 3215–3222. AAAI Press, 2018. doi: 10.1609/aaai.v32i1.11796. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11796>.
- [5] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed prioritized experience replay. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=H1Dy--0Z>.
- [6] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.
- [7] Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=r1lyTjAqYX>.
- [8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [9] Thibault Lahire, Matthieu Geist, and Emmanuel Rachelson. Large batch experience replay. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 11790–11813. PMLR, 2022. URL <https://proceedings.mlr.press/v162/lahire22a.html>.
- [10] Ang A. Li, Zongqing Lu, and Chenglin Miao. Revisiting prioritized experience replay: A value perspective. *arXiv preprint arXiv:2102.03261*, 2021. URL <https://arxiv.org/abs/2102.03261>.
- [11] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. doi: 10.1007/BF00992699.
- [12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [13] Johan Samir Obando-Ceron and Pablo Samuel Castro. Revisiting Rainbow: Promoting more insightful and inclusive deep reinforcement learning research. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 1373–1383. PMLR, 2021. URL <https://proceedings.mlr.press/v139/ceron21a.html>.

- [14] Yangchen Pan, Jincheng Mei, Amir-massoud Farahmand, Martha White, Hengshuai Yao, Mohsen Rohani, and Jun Luo. Understanding and mitigating the limitations of prioritized experience replay. In James Cussens and Kun Zhang, editors, *Proceedings of the 38th Conference on Uncertainty in Artificial Intelligence*, volume 180 of *Proceedings of Machine Learning Research*, pages 1561–1571. PMLR, 2022. URL <https://proceedings.mlr.press/v180/pan22a.html>.
- [15] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016. URL <https://arxiv.org/abs/1511.05952>.
- [16] Shivakanth Sujit, Somjit Nath, Pedro Braga, and Samira Ebrahimi Kahou. Prioritizing samples in reinforcement learning with reducible loss. In *Advances in Neural Information Processing Systems*, volume 36, pages 23237–23258, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/48726631f87322012c6be38e00c72a47-Abstract-Conference.html.
- [17] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.
- [18] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, pages 6306–6315, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html.
- [19] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100. AAAI Press, 2016. doi: 10.1609/aaai.v30i1.10295. URL <https://ojs.aaai.org/index.php/AAAI/article/view/10295>.
- [20] Kenny Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments. *arXiv preprint arXiv:1903.03176*, 2019. URL <https://arxiv.org/abs/1903.03176>.
- [21] Shangdong Zhang and Richard S. Sutton. A deeper look at experience replay. *arXiv preprint arXiv:1712.01275*, 2017. URL <https://arxiv.org/abs/1712.01275>.

A Code and reproducibility

Code and experiment scripts are available at <https://anonymous.4open.science/r/bias-experience-replay/>.

B Additional derivations

B.1 Effective sampling distribution of SAMPLE

Let A denote the PER-sampled anchor index and J denote the index actually used for training after sibling replacement. SAMPLE first draws

$$\Pr(A = i) = \frac{u_i}{S},$$

then draws J uniformly from the anchor group $I(\kappa(A))$. A transition j can be used for training whenever the anchor lies in the same group as j , hence

$$q_{\text{SAMPLE}}(j) = \Pr(J = j) = \sum_{i \in I(\kappa(j))} \frac{u_i}{S} \frac{1}{n_{\kappa(j)}} = \frac{s_{\kappa(j)}}{S n_{\kappa(j)}}.$$

369 The uniform-buffer probability is $1/N$, so the finite-buffer correction ratio is

$$\rho_j = \frac{1/N}{q_{\text{SAMPLE}}(j)} = \frac{n_{\kappa(j)} S}{N s_{\kappa(j)}}.$$

370 With full correction, fixed priorities, and fixed per-transition losses, the expected weighted estimator
 371 matches the uniform-buffer estimator. With partial correction, residual weighting remains across
 372 groups through s_g , but the conditional distribution inside each selected exact group is uniform.

373 **B.2 Bias of PER’s within-group target**

374 For a fixed group g , let Y_i denote the Bellman target of sibling i . The empirical conditional target is

$$\bar{Y}_g = \frac{1}{n_g} \sum_{i \in I(g)} Y_i,$$

375 whereas PER uses the priority-weighted conditional expectation

$$\tilde{Y}_g^{\text{PER}} = \sum_{i \in I(g)} \frac{u_i}{s_g} Y_i.$$

376 The finite-buffer within-group bias is therefore

$$B_g^{\text{PER}} = \tilde{Y}_g^{\text{PER}} - \bar{Y}_g = \sum_{i \in I(g)} \left(\frac{u_i}{s_g} - \frac{1}{n_g} \right) Y_i.$$

377 This term is zero if priorities are constant within the group or if the priority deviations are uncor-
 378 related with sibling targets in the finite buffer. It can be large when rare stochastic outcomes have
 379 systematically larger TD errors.

380 **B.3 Variance of SAMPLE and AVG**

381 Condition on the selected group g and fixed targets. SAMPLE draws one sibling uniformly, so its
 382 conditional target variance is the empirical finite-population variance of the sibling targets. If AVG
 383 samples K siblings with replacement, then

$$\text{Var}(\hat{Y}_{g,K} \mid g) = \frac{\sigma_g^2}{K},$$

384 where σ_g^2 is the variance of a uniformly sampled sibling target. If it samples without replacement, the
 385 finite-population correction gives

$$\text{Var}(\hat{Y}_{g,K} \mid g) = \frac{\sigma_g^2}{K} \left(1 - \frac{K-1}{n_g-1} \right), \quad K \leq n_g.$$

386 Thus AVG has the same conditional expectation as SAMPLE and lower fixed-target conditional vari-
 387 ance. In deep bootstrapped learning this variance argument does not by itself determine performance,
 388 because target networks, priority updates, stale priorities, and optimizer dynamics also change.

389 **B.4 MODEL targets**

390 MODEL stores a counter over full outcomes

$$Z = (s', r, d_{\text{term}}, d_{\text{trunc}})$$

391 for each exact (s, a) group. Sampling from this joint counter is necessary because rewards, next
 392 states, and terminations can be dependent. A model that samples marginal next states and averaged
 393 rewards can create impossible synthetic outcomes and does not implement the empirical conditional
 394 outcome law used in the theorem-level method.

C Experimental details

Loss and optimizer. All reported DQN variants use Double-DQN targets [19], Adam [8], and mean-squared TD error. We use MSE rather than Huber loss [6] because the central phenomenon is driven by high-magnitude stochastic outcomes: clipping the loss changes the tail signal itself. In preliminary OutlierBandit checks, Huber loss failed to expose the value-estimation pathology, while MSE with Adam followed the same practical lesson emphasized by modern Rainbow re-evaluations that loss and optimizer choices can materially change conclusions [13]. We therefore keep the loss fixed across methods and compare replay distributions.

MinAtar protocol and tuning. The MinAtar network and training scale follow the compact MinAtar protocol [20]: two convolutional layers with 16 and 32 channels, a 128-unit fully connected layer, $\gamma = 0.99$, sticky-action probability 0.1, replay capacity 100,000, minibatch size 32, target-network update interval 1000, and 5 million environment steps. Epsilon decays to 0.1 over 100,000 steps. PER uses $\alpha = 0.6$ and anneals β over training.

Learning rates were selected by a clean-condition AUC sweep and then reused for both clean and MP-TAIL. The executed grid was $1.2 \cdot 10^{-4}$, $6 \cdot 10^{-5}$, $3 \cdot 10^{-5}$, $7.5 \cdot 10^{-6}$, with additional low-rate checks at $3.75 \cdot 10^{-6}$ and $1.875 \cdot 10^{-6}$ when PER favored smaller rates. The final selected settings were expanded to five seeds.

Table 3: Selected MinAtar learning rates used for the reported clean and MP-TAIL runs.

Game	DDQN lr	PER lr	SAMPLE lr
Asterix	7.5e-06	1.88e-06	7.5e-06
Breakout	6e-05	3e-05	6e-05
Freeway	7.5e-06	1.88e-06	7.5e-06
Seaquest	7.5e-06	3.75e-06	7.5e-06
SpaceInv.	3e-05	7.5e-06	3e-05

FrozenLake schedules. For H100/H300, the intended action is taken with probability 0.99, with the two side slips sharing the remaining probability. Rewards are +100 for the goal, −300 for holes, and −0.01 per step. The milder H50 variant uses +50, −100, and −0.01. Exact groups are true (s, a) pairs. Theoretical finite-buffer calculations identify a worst-state PER target bias of about −50.4 in H100/H300 and −15.3 in H50.

D VQ-VAE grouping

For MinAtar, we trained a separate VQ-VAE per game and used the frozen encoder to define approximate state-action groups. The encoder maps each observation to a discrete code grid; this code grid is packed into a key and paired with the action to define $(\phi(s), a)$ groups. The VQ-VAE was trained only to define groups, not jointly with the RL objective.

The selection criterion was support. We compared grid sizes 1×1 and 2×1 and codebook sizes including 8, 16, 32, and 256 where relevant. The useful abstraction must avoid both failure modes: if it is too coarse, it merges states with different conditional outcomes; if it is too fine, most buckets become singleton or very small and sibling replacement has little support. The selected 2×1 , $K = 16$ abstractions provide good support in four games. SpaceInvaders uses $K = 32$ because the later support sweep showed that $K = 16$ was too coarse, while $K = 32$ kept adequate bucket support.

E Additional results

Clean MinAtar is a control condition. PER is not uniformly worse there, and SAMPLE does not uniformly dominate DDQN. This is important for interpretation: the method is aimed at stochastic-tail self-selection, not at replacing PER in every regime. FrozenLake-H50 shows the same diagnostic PER pathology but DDQN has the best AUC, so the main text centers H100/H300. AVG-g4 is included in the released tables and figures but is not emphasized because it underperforms AVG-g2 in FrozenLake.

Table 4: Selected VQ-VAE support diagnostics from the W&B VQ-VAE training project. Pair coverage is the fraction of possible code-action buckets observed in the diagnostic sample.

Game	Grid, K	recon.	perp.	pair cov.	median bucket	frac ≥ 100
Asterix	$2 \times 1, 16$	0.061	14.0	0.664	38	0.822
Breakout	$2 \times 1, 16$	0.028	8.7	0.133	156	0.973
Freeway	$2 \times 1, 16$	0.090	15.1	0.951	33	0.677
Seaquest	$2 \times 1, 16$	0.024	12.1	0.569	44	0.767
SpaceInv.	$2 \times 1, 32$	0.082	16.8	0.158	41	0.760

Table 5: Clean MinAtar AUC return, mean $\pm$ standard error over five seeds.

Game	DDQN	PER	SAMPLE
Asterix	23.70 ± 0.46	22.97 ± 1.34	23.39 ± 0.50
Breakout	17.23 ± 0.62	19.17 ± 0.35	19.24 ± 0.51
Freeway	52.97 ± 0.15	49.80 ± 0.34	51.70 ± 0.11
Seaquest	32.63 ± 2.11	34.02 ± 2.27	35.57 ± 1.84
SpaceInv.	83.39 ± 3.02	67.92 ± 3.30	79.18 ± 1.45

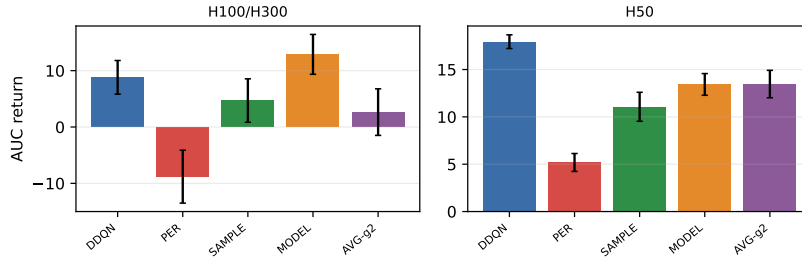


Figure 7: FrozenLake AUC return for both selected reward schedules. H100/H300 is the sharper exact-state stress test; H50 is a milder robustness setting.